



Documentazione

SlotLab

Sistema per la prenotazione nominativa degli spazi universitari

Progetto di Programmazione Web e Mobile

Emanuel Tomasello - Francesco Genova

Universita degli Studi di Palermo

Anno accademico 2025/2026

Indice

1. Introduzione	5
1.1. Descrizione del sistema	5
1.2. Scopo e obiettivi	5
1.3. Architettura del sistema	5
1.4. Attori principali	6
1.4.1. Visitatore non autenticato	6
1.4.2. Utente	6
1.4.3. Amministratore	6
1.4.4. DBMS e servizi infrastrutturali	6
1.5. Ambienti utilizzati	6
2. Componenti del sistema	6
2.1. Componenti e librerie Ionic	6
2.2. Componenti e librerie Angular	7
2.3. Componenti custom	7
3. Interfacce e modelli custom	7
3.1. Valori di dominio	8
4. Services	8
4.1. Auth	8
4.2. RefreshSession	8
4.3. AuthInterceptor	8
4.4. ThemeService	9
4.5. Utility HTTP e immagini	9
4.6. Moduli backend di dominio	9
5. API e servizi esterni	9
5.1. OpenStreetMap	9
5.2. SMTP	9
5.3. Fonti dei dati del campus	9
5.4. API interne	10
6. Persistenza dei dati	10
6.1. Frontend	10
6.2. Backend	10
6.2.1. Introduzione	10

6.2.2. Schema logico del database	10
6.2.3. Descrizione delle tabelle	11
6.2.4. Migrazioni e seed	11
7. Middleware utilizzati	11
7.1. Parser JSON di Express	11
7.2. requireAuth e requireRole	12
7.3. AuthInterceptor e guardie Angular	12
7.4. Protezione delle richieste cookie	12
7.5. Rate limiting	12
7.6. Validazione upload	12
7.7. Gestori 404 e degli errori	12
8. Rotte utilizzate	12
8.1. Introduzione	12
8.2. Rotte di accesso e account	13
8.3. Catalogo e check-in	13
8.4. Prenotazioni e preferiti	13
8.5. Segnalazioni e notifiche	14
8.6. Rotte amministrative	14
9. Funzionalità	14
9.1. FSM delle pagine	14
9.1.1. Flusso utente	15
9.1.2. Flusso amministratore	15
9.2. Home e ricerca spazi	15
9.3. Gestione account	16
9.3.1. Registrazione e login	16
9.3.2. Password dimenticata	16
9.3.3. Profilo	16
9.4. Gestione prenotazioni	16
9.4.1. Scelta della fascia	16
9.4.2. Partecipanti e conferma	16
9.4.3. Dettaglio, cancellazione e notifiche	16
9.5. Check-in tramite QR	16
9.6. Preferiti	16
9.7. Segnalazioni	17
9.8. Notifiche e avvisi	17
9.9. Gestione amministrativa	17
9.9.1. Edifici e spazi	17

9.9.2. Disponibilit� e indisponibilit�	17
9.9.3. Prenotazioni, utenti e segnalazioni	17
9.9.4. Statistiche	17
10. Sicurezza, configurazione e deployment	18
10.1. Password e sessioni	18
10.2. Variabili di ambiente	18
10.3. Sviluppo, test e produzione	18
10.4. Comandi principali	19
11. Verifica e qualita	19
11.1. Strategia di test	19
11.2. Collaudo manuale	19
11.3. Responsive design	19
11.4. Limiti operativi dichiarati	19
12. Conclusioni	20
12.1. Riferimenti interni	20

1. Introduzione

1.1. Descrizione del sistema

SlotLab è una web application responsive dedicata agli spazi dell'Università degli Studi di Palermo nel campus di Viale delle Scienze. Il sistema permette agli studenti registrati di individuare aule studio, laboratori e sale riunioni, conoscere servizi e posti disponibili, prenotare posti nominativi e registrare la propria presenza tramite il QR permanente associato allo spazio.

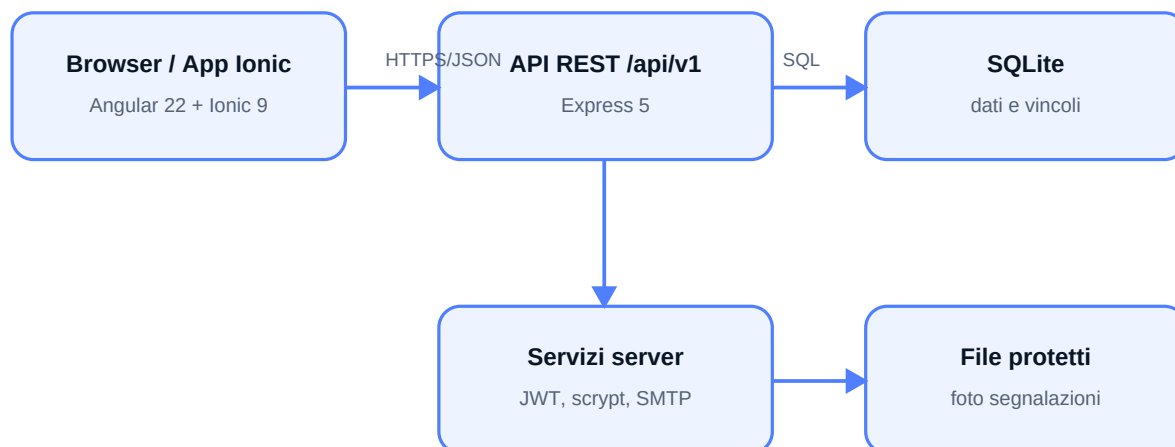
L'applicazione comprende anche un'area amministrativa. Gli amministratori gestiscono edifici, spazi, capienze, fasce ricorrenti, indisponibilità eccezionali, utenti, prenotazioni, segnalazioni, avvisi e indicatori statistici. Tutti i dati visualizzati nei flussi operativi provengono dalle API Express e dal database SQLite.

1.2. Scopo e obiettivi

- **Centralizzare la disponibilità:** riunire in un solo catalogo gli spazi del campus e le relative fasce prenotabili.
- **Usare posti nominativi:** ogni organizzatore e partecipante occupa un posto associato a un account.
- **Prevenire conflitti:** controllare capienza, sovrapposizioni personali, limite di cinque prenotazioni future e anticipo minimo di un'ora.
- **Registrare la presenza:** effettuare il check-in personale attraverso il QR dello spazio nella finestra temporale autorizzata.
- **Supportare la gestione:** fornire strumenti amministrativi coerenti con il catalogo utente e notificare le modifiche che incidono sulle prenotazioni.
- **Misurare l'utilizzo:** calcolare statistiche su fasce terminate usando snapshot storici della capienza offerta.
- **Garantire sicurezza e continuità:** proteggere sessioni, password, allegati e ruoli senza memorizzare segreti nel frontend.

1.3. Architettura del sistema

L'architettura è di tipo client-server. Il client Angular/Ionic comunica in JSON con un backend REST Express sotto il prefisso **/api/v1**. Express applica autenticazione, validazione e regole transazionali, quindi legge o modifica SQLite. Le foto delle segnalazioni sono salvate in una cartella server non pubblica e restituite solo tramite endpoint autenticati. Il recupero password usa un relay SMTP configurabile.



1.4. Attori principali

1.4.1. Visitatore non autenticato

Puo registrare un account, eseguire il login e richiedere il recupero password. Le route applicative sono protette e riportano al login conservando, quando previsto, la destinazione richiesta.

1.4.2. Utente

Consulta Home, mappa, catalogo e dettagli; gestisce preferiti e prenotazioni; aggiunge partecipanti tramite email; esegue il check-in; invia segnalazioni con foto; legge notifiche e modifica foto o password del proprio profilo. Puo eliminare definitivamente il proprio account reinserendo la password.

1.4.3. Amministratore

Gestisce le risorse del campus e consulta dati operativi e storici. Non prenota come studente e non modifica password o ruolo degli altri utenti. Gli amministratori sono creati dal seed tecnico e non dalla registrazione pubblica.

1.4.4. DBMS e servizi infrastrutturali

SQLite garantisce integrita referenziale, unicità e vincoli di dominio. Il server SMTP consegna le password temporanee del recupero. OpenStreetMap fornisce le tile cartografiche visualizzate da Leaflet.

1.5. Ambienti utilizzati

Ambito	Tecnologia	Ruolo
Frontend	Angular 22.0.1, Ionic Angular 9, TypeScript 6, RxJS 7.8	SPA responsive, componenti, routing e chiamate HTTP
Backend	Node.js, Express 5.2.1, CommonJS	API REST, regole applicative e integrazione servizi
Persistenza	SQLite 3 tramite sqLite3 6.0.1	Database relazionale locale e transazionale
Sicurezza	jsonwebtoken 9.0.3, crypto/scrypt, cookie-parser 1.4.7	JWT, refresh token, hashing e cookie HttpOnly
Email	Nodemailer 10	Relay SMTP configurabile
UI e device	Leaflet 1.9.4, Ionicons 8, Capacitor 8	Mappa, icone e integrazione mobile
Test	Node test runner, Vitest/Angular TestBed, jsdom	Test backend e frontend isolati

2. Componenti del sistema

2.1. Componenti e librerie Ionic

SlotLab usa i componenti standalone di Ionic per l'involucro delle pagine e l'integrazione con il browser mobile. I principali elementi sono:

- **IonApp**: contenitore radice dell'applicazione.
- **IonContent**: area scorrevole delle pagine con variabili di tema condivise.
- **IonIcon**: icone di navigazione, servizi, azioni e stati.
- **IonRouterOutlet**: caricamento delle route Angular all'interno dell'app.

- **Capacitor**: base per l'esecuzione e l'integrazione su dispositivi mobili.

2.2. Componenti e librerie Angular

- **Router e guardie funzionali**: separano visitatori, utenti e amministratori e conservano la destinazione del check-in.
- **HttpClient e interceptor**: inviano richieste REST, aggiungono il Bearer token e coordinano il rinnovo della sessione.
- **Signals**: espongono identità, stato di autenticazione, logout e tema in modo reattivo.
- **FormsModule**: collega input, select, checkbox e calendari ai dati dei form.
- **Lazy loading**: carica ogni pagina soltanto quando viene visitata.
- **RxJS**: coordina richieste, cancellazione delle subscription e aggiornamento asincrono.

2.3. Componenti custom

Componente	Funzione
TopbarComponent	Navigazione desktop utente, identità, notifiche e tema.
MobileNavigationComponent	Barra inferiore responsive per le principali route utente e admin.
AdminSidebarComponent / BookingsSidebarComponent	Navigazione laterale amministrativa e collegamento al profilo.
CampusMapComponent	Mappa Leaflet, marker degli edifici e caricamento degli spazi selezionati.
SpaceCard / SpaceListItem	Sintesi dello spazio, immagine, stato e gestione preferito.
FavoriteSpaceCard	Card della raccolta personale con rimozione reattiva.
SpaceSearch / QuickFilters	Ricerca e filtri disponibili ora, accessibili e posti minimi.
NextBookingComponent	Mostra la prenotazione futura più vicina e apre il suo dettaglio.
BookingListItem / BookingDetail	Elenco, stato, cancellazione e partecipanti della prenotazione.
ReportListItem / ReportDetail	Riepilogo, foto protetta e stato della segnalazione.
SpaceQrComponent	Genera il QR permanente con route /check-in/{spaceId}.
ThemeToggleComponent	Seleziona e persiste tema chiaro o scuro.

3. Interfacce e modelli custom

Le interfacce TypeScript descrivono il contratto usato dai componenti. I payload HTTP usano camelCase; le colonne SQLite usano snake_case e vengono mappate dal backend.

Modello	Campi principali e utilizzo
Account	id, firstName, lastName, email, role, createdAt; identità autenticata.
Credentials / Registration	email, password e, per la registrazione, nome e cognome.
RefreshedSession	accessToken, expiresIn e dati utente dopo il refresh.
SpaceSummary / Space	id, nome, edificio, piano, tipo, capienza, accessibilità, stato, servizi e immagine.

Modello	Campi principali e utilizzo
CampusBuilding	id, numero ufficiale, nome, indirizzo e coordinate.
BookingSummary / BookingListItem	id, spazio, data, orari, stato, edificio e numero partecipanti.
BookingGroup	prenotazione con informazioni di organizzatore e partecipanti.
FavoriteSpace	estensione dello spazio con stato preferito aggiornato.
SpaceReport	categoria, descrizione, priorit�, stato, foto e date.
AdminBooking / AdminReport	viste amministrative complete di partecipanti, presenze e autore.
StatisticsResponse	totali, tassi, serie temporali, graduatorie e distribuzioni.

3.1. Valori di dominio

Dominio	Valori
Ruolo	user, admin
Tipo spazio	study_room, laboratory, meeting_room
Stato spazio	active, maintenance, deactivated
Servizio	wifi, power_outlets, projector, computer, air_conditioning
Stato prenotazione	confirmed, completed
Ruolo partecipante	organizer, participant
Categoria segnalazione	technical, accessibility, cleaning, other
Stato segnalazione	open, in_progress, resolved

4. Services

4.1. Auth

Il servizio Auth conserva l'access token soltanto in memoria e pubblica l'utente corrente tramite signal. Implementa registrazione, login, recupero della sessione, logout e scelta della destinazione iniziale in base al ruolo. Il refresh token non   accessibile al codice Angular perche risiede nel cookie HttpOnly.

4.2. RefreshSession

Coordina il rinnovo tramite **POST /auth/refresh**. Se piu richieste ricevono 401 contemporaneamente, condivide un solo refresh e ripete le richieste dopo avere ottenuto il nuovo access token. Durante il logout blocca nuovi rinnovi e attende quello eventualmente gia in corso.

4.3. AuthInterceptor

Aggiunge **Authorization: Bearer** alle API protette, esclude le chiamate pubbliche e gestisce un tentativo controllato di refresh. Non salva token in localStorage e non modifica richieste esterne.

4.4. ThemeService

Applica la classe Ionic del tema al documento e persiste soltanto la preferenza **slotlab-theme** in localStorage. Se la memoria non è disponibile, l'interfaccia continua a funzionare con il tema corrente.

4.5. Utility HTTP e immagini

La funzione **allPages** attraversa risposte paginate quando una schermata necessita dell'insieme completo. Le funzioni per le immagini associano una risorsa standard alla tipologia dello spazio e producono URL protetti per foto profilo e segnalazioni.

4.6. Moduli backend di dominio

Modulo	Responsabilità
password / validation	Validazione condivisa e hash script con confronto timing-safe.
tokens	Creazione JWT HS256, refresh casuale e hash SHA-256 lato server.
account-password / delete-account	Cambio password, revoca sessione e cancellazioni coordinate.
recovery	Generazione della nuova password e invio asincrono via SMTP.
profile-photo / report-files	Verifica del contenuto reale e consegna autenticata delle immagini.
time	Regole temporali nel fuso Europe/Rome.
occupied-seats	Conteggio atomico dei posti impegnati per spazio e fascia.
occurrences / statistics	Consolidamento delle fasce terminate e calcolo degli indicatori.
transaction	Wrapper Promise e transazioni SQLite BEGIN IMMEDIATE.

5. API e servizi esterni

5.1. OpenStreetMap

Leaflet carica le tile da **tile.openstreetmap.org** e mostra l'attribuzione OpenStreetMap. Le coordinate degli edifici sono conservate nel database SlotLab; OpenStreetMap non riceve dati personali o prenotazioni.

5.2. SMTP

Il recupero password usa Nodemailer e un server SMTP scelto in fase di installazione. Host, porta, modalità TLS, mittente, eventuali credenziali e certificato CA sono letti da variabili locali. In assenza di configurazione SMTP il server può funzionare, ma il recupero email non è operativo.

5.3. Fonti dei dati del campus

Gli edifici del campus sono dati amministrativi iniziali, ricavati dalle fonti UniPa e inseriti nel catalogo locale. Non esiste una dipendenza runtime da un'API UniPa: la disponibilità viene determinata interamente dalle configurazioni SlotLab.

5.4. API interne

Il frontend usa esclusivamente l'API REST di SlotLab. In sviluppo Angular inoltra **/api/**** a Express tramite proxy; in produzione frontend e backend devono essere esposti sullo stesso dominio HTTPS tramite reverse proxy.

6. Persistenza dei dati

6.1. Frontend

L'identità e l'access token rimangono in memoria. Il refresh token rimane nel cookie Secure e HttpOnly. localStorage conserva soltanto la preferenza del tema. Gli elenchi vengono ricaricati dalle API quando una pagina torna attiva, evitando che una precedente simulazione locale diventi la fonte del dato.

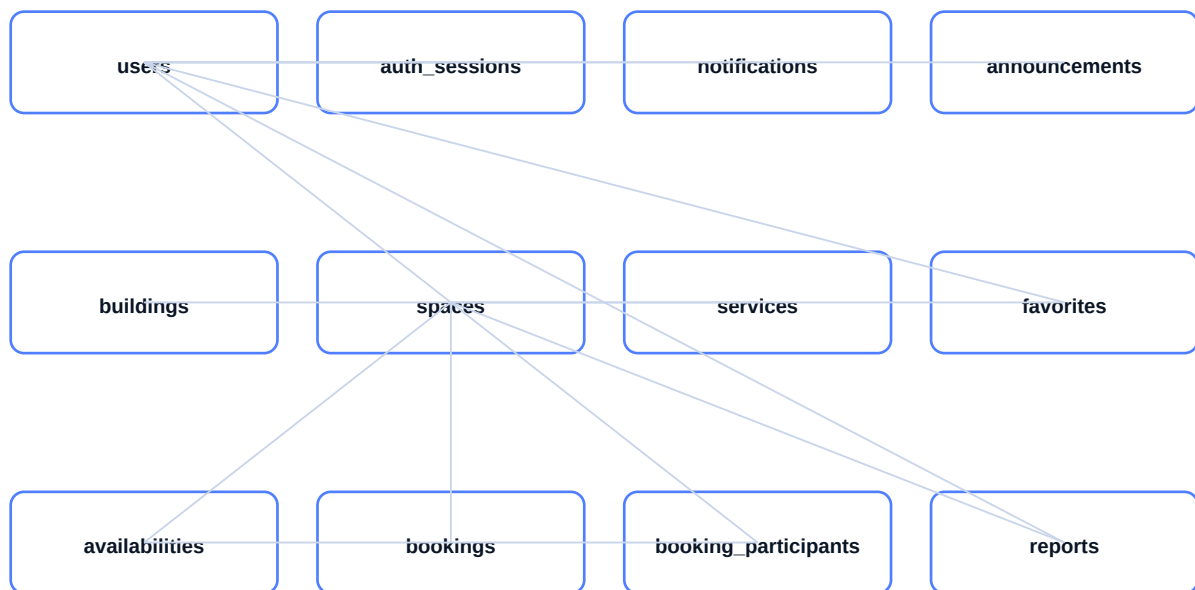
6.2. Backend

6.2.1. Introduzione

SQLite è aperto da una connessione condivisa con foreign key abilitate e busy timeout. Le migrazioni sono versionate e accompagnate da checksum: un file già applicato non può essere modificato silenziosamente. Le operazioni che coinvolgono capienza, prenotazioni, notifiche e cancellazioni usano transazioni.

6.2.2. Schema logico del database

Schema logico semplificato - relazioni principali



Altre tabelle: space_services, unavailabilities, slot_occurrences, booking_requests, file_deletions

6.2.3. Descrizione delle tabelle

Tabella	Contenuto
buildings	Numero ufficiale, nome, indirizzo e coordinate degli edifici.
spaces	Spazi, tipologia, capienza, accessibilit� e stato.
services / space_services	Catalogo dei servizi e relazione molti-a-molti con gli spazi.
availabilities	Fasce settimanali ricorrenti con periodo e stato di ritiro.
unavailabilities	Eccezioni per data, intervallo e motivo amministrativo.
slot_occurrences	Snapshot delle fasce concrete e della capienza offerta per le statistiche.
users	Account normalizzati, ruolo e hash della password; la migrazione 5 aggiunge la foto profilo.
auth_sessions	Una sessione attiva per account, hash refresh, jti e scadenze.
bookings	Prenotazione di una fascia per uno spazio.
booking_participants	Organizzatore e partecipanti, presenza e data del check-in.
booking_requests	Chiavi di idempotenza e impronta del payload.
favorites	Relazione univoca utente-spazio.
reports	Segnalazioni, priorit� automatica, stato e nome file della foto.
announcements / notifications	Avvisi globali, notifiche personali e stato di lettura.
file_deletions	Coda tecnica per coordinare eliminazioni di file e transazioni dati.

6.2.4. Migrazioni e seed

- Migrazione 1: schema iniziale delle entit  applicative.
- Migrazione 2: integrit  completa degli snapshot delle occorrenze.
- Migrazione 3: supporto alla cancellazione coordinata dei file.
- Migrazione 4: tracciamento storico delle occorrenze.
- Migrazione 5: foto profilo memorizzate come dato binario e tipo MIME.
- Il seed principale inserisce servizi, edifici/spazi iniziali e l'amministratore tecnico in modo idempotente.
- Il seed demo richiede un database separato, crea tre fasce giornaliere per ogni spazio e dati statistici ripetibili.

7. Middleware utilizzati

7.1. Parser JSON di Express

Accetta i payload JSON e applica i limiti configurati. Corpi malformati producono **INVALID_JSON**; gli errori interni non espongono stack o query al client.

7.2. requireAuth e requireRole

requireAuth verifica firma HS256, scadenza, claim obbligatori, esistenza dell'account e corrispondenza del jti con la sessione corrente. requireRole autorizza solo i ruoli ammessi dalla rotta. Gli errori sono 401 per autenticazione e 403 per ruolo insufficiente.

7.3. AuthInterceptor e guardie Angular

L'interceptor aggiunge il Bearer e orchestra il refresh. sessionGuard protegge route autenticate, guestGuard evita i form pubblici a chi ha già una sessione e adminGuard richiede il ruolo amministratore. Le guardie migliorano la navigazione, mentre l'autorizzazione effettiva resta sempre nel backend.

7.4. Protezione delle richieste cookie

Refresh e logout richiedono l'header **X-SlotLab-Request: 1**, cookie valido e origine coerente. La protezione rifiuta richieste cross-site e non richiede CORS aperto.

7.5. Rate limiting

Le operazioni sensibili, tra cui account e check-in, hanno limiti per utente o indirizzo. Dietro Nginx il trust proxy può essere impostato soltanto su loopback, in modo che Express non accetti intestazioni client arbitrarie.

7.6. Validazione upload

Le foto profilo accettano JPEG, PNG o WebP fino a 2 MB. Le segnalazioni accettano una foto degli stessi formati fino a 5 MB. Il backend controlla la firma binaria, genera il nome del file e non usa percorsi forniti dal client.

7.7. Gestori 404 e degli errori

Le route sconosciute producono **ROUTE_NOT_FOUND**. Tutti gli errori applicativi usano la forma **{ error: { code, message } }**; gli errori imprevisti diventano **INTERNAL_ERROR**.

8. Rotte utilizzate

8.1. Introduzione

Tutte le rotte applicative hanno prefisso **/api/v1**. Le risposte usano JSON camelCase; gli endpoint foto restituiscono il contenuto binario protetto. Le tabelle seguenti riportano le rotte presenti nel backend attuale.

8.2. Rotte di accesso e account

Metodo e rotta	Accesso	Funzione
POST /auth/register	Pubblico	Crea un account user validato.
POST /auth/login	Pubblico	Verifica credenziali, crea JWT e cookie refresh.
POST /auth/refresh	Cookie protetto	Ruota sessione e restituisce un nuovo access token.
POST /auth/logout	Cookie protetto	Elimina la sessione corrente in modo idempotente.
POST /auth/forgot-password	Pubblico	Genera e invia una nuova password senza rivelare l'esistenza dell'email.
GET /users/me	user/admin	Restituisce il profilo corrente.
GET PUT DELETE /users/me/photo	user/admin	Legge, sostituisce o elimina la foto profilo.
PATCH /users/me/password	user/admin	Cambia password e invalida la sessione.
DELETE /users/me	user	Elimina definitivamente il proprio account.

8.3. Catalogo e check-in

Metodo e rotta	Funzione
GET /buildings	Elenca gli edifici per numero ufficiale.
GET /buildings/{id}	Restituisce il dettaglio edificio.
GET /buildings/{id}/spaces	Elenca gli spazi del marker selezionato.
GET /spaces	Ricerca e filtra il catalogo con paginazione.
GET /spaces/recommended	Restituisce al massimo tre consigli per oggi o domani.
GET /spaces/{id}	Dettaglio, servizi e stato preferito.
GET /spaces/{id}/availability?date=...	Fasce, posti liberi e motivi di non prenotabilità.
POST /spaces/{id}/check-in	Registra la presenza nella finestra autorizzata.

8.4. Prenotazioni e preferiti

Metodo e rotta	Funzione
GET /bookings	Prenotazioni correnti dell'utente.
GET /bookings/{id}	Dettaglio e partecipanti per un utente coinvolto.
POST /bookings	Crea atomicamente una prenotazione con Idempotency-Key.
POST /bookings/{id}/participants	Aggiunge un account esistente tramite email.
DELETE /bookings/{id}/participants/{participantId}	Rimuove un partecipante o consente l'abbandono.
DELETE /bookings/{id}	Cancella il gruppo e notifica gli altri partecipanti.
GET POST DELETE /favorites/{spaceId}	Elenca, aggiunge o rimuove preferiti senza duplicati.

8.5. Segnalazioni e notifiche

Metodo e rotta	Funzione
GET /reports	Elenca le segnalazioni dell'autore.
GET /reports/{id}	Restituisce il dettaglio all'autore.
GET /reports/{id}/photo	Restituisce la foto protetta all'autore.
POST /spaces/{id}/reports	Crea una segnalazione multipart e assegna la priorita.
GET /notifications	Elenca notifiche personali paginate.
PATCH /notifications/{id}/read	Segna una notifica come letta.
PATCH /notifications/read-all	Segna tutte le notifiche come lette.

8.6. Rotte amministrative

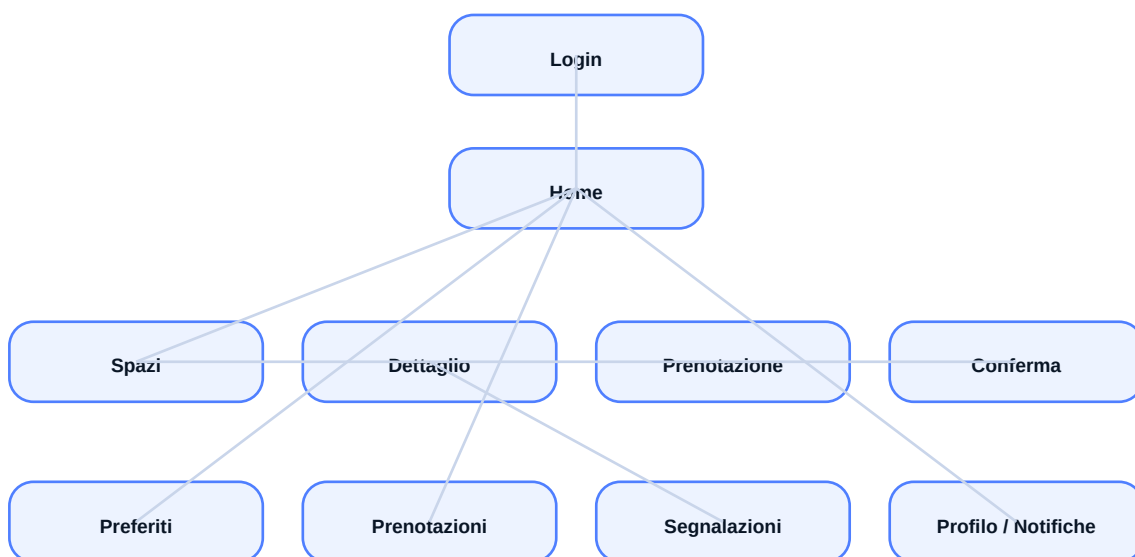
Gruppo	Rotte principali
Panoramica	GET /admin/summary
Edifici	GET, POST /admin/buildings; GET, PATCH, DELETE /admin/buildings/{id}
Spazi	GET, POST /admin/spaces; GET, PATCH, DELETE /admin/spaces/{id}
Disponibilita	GET, POST /admin/spaces/{id}/availability; PATCH, DELETE con availabilityId
Indisponibilita	GET, POST /admin/spaces/{id}/unavailability; PATCH, DELETE con unavailabilityId
Prenotazioni	GET /admin/bookings con filtri e dettaglio partecipanti
Segnalazioni	GET /admin/reports; GET foto; PATCH stato
Utenti	GET /admin/users; GET e DELETE /admin/users/{id}
Avvisi	POST /admin/announcements
Statistiche	GET /admin/statistics?dateFrom=...&dateTo;=...

9. Funzionalità

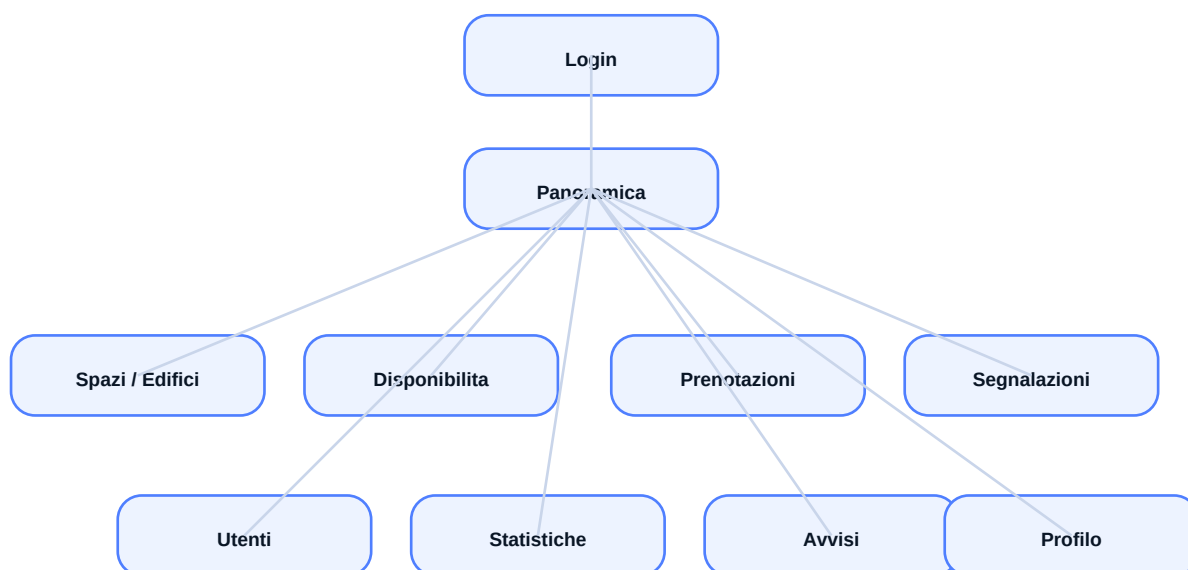
9.1. FSM delle pagine

Le due macchine a stati seguenti sintetizzano la navigazione principale. La sessione determina il ramo utente o amministratore; logout e revoca riportano al login.

9.1.1. Flusso utente



9.1.2. Flusso amministratore



9.2. Home e ricerca spazi

La Home carica al massimo tre consigli basati sulla prima fascia prenotabile e mostra la prenotazione futura più vicina. La mappa presenta i marker degli edifici e interroga l'API quando ne viene selezionato uno. "Vedi tutti" apre il catalogo completo.

La pagina Spazi ricerca per nome dello spazio, nome o numero dell'edificio. I filtri disponibili sono Disponibili ora, Accessibili e Posti almeno 10. Il calcolo dei posti usa le prenotazioni reali, non la capienza nominale isolata. Gli spazi in manutenzione o disattivati rimangono visibili ma non prenotabili.

9.3. Gestione account

9.3.1. Registrazione e login

La registrazione normalizza l'email, valida i dati e assegna sempre il ruolo user. Dopo la creazione viene richiesto il login. Il login verifica l'hash script, sostituisce l'eventuale sessione precedente e restituisce un access token di 30 minuti; il refresh di 7 giorni resta nel cookie.

9.3.2. Password dimenticata

Per una email sintatticamente valida l'interfaccia mostra sempre lo stesso esito. Se l'account esiste e SMTP è configurato, il server genera una nuova password conforme, salva solo il suo hash, invalida la sessione e invia la password via email.

9.3.3. Profilo

Utenti e amministratori possono cambiare password, foto profilo e tema. L'utente può inoltre aprire Le mie segnalazioni ed eliminare il proprio account. L'avatar stesso apre il selettore immagine; la rimozione ripristina le iniziali.

9.4. Gestione prenotazioni

9.4.1. Scelta della fascia

Dal dettaglio spazio l'utente apre Nuova prenotazione, sceglie una data entro 30 giorni e una fascia configurata dall'admin. Il calendario e l'intero controllo sono cliccabili. Il backend esclude fasce ritirate, indisponibili, piene, iniziate o con meno di un'ora di anticipo.

9.4.2. Partecipanti e conferma

I pulsanti meno e più determinano il numero di campi email. Le email devono appartenere ad account normali, non possono essere duplicate e non possono coincidere con l'organizzatore. La transazione controlla capienza, sovrapposizioni e limite di cinque prenotazioni per ogni persona. La conferma mostra il codice numerico reale e i dati salvati.

9.4.3. Dettaglio, cancellazione e notifiche

L'organizzatore aggiunge o rimuove partecipanti fino a un'ora prima. Un partecipante può abbandonare soltanto la propria partecipazione; l'organizzatore cancella l'intera prenotazione. Le cancellazioni eliminano il record, liberano posti e notificano gli altri interessati. Le prenotazioni concluse restano consultabili solo dall'admin.

9.5. Check-in tramite QR

Il QR permanente dello spazio apre `/check-in/{spaceId}`. Il server cerca una partecipazione nello stesso spazio e consente il check-in da 15 minuti prima a 30 minuti dopo l'inizio. Un secondo tentativo conserva il primo orario. Il QR non è mostrato all'utente nella pagina di dettaglio: è destinato all'affissione fisica nell'aula e alla gestione admin.

9.6. Preferiti

Ogni utente può salvare più spazi, anche se pieni o non attivi. Il vincolo univoco utente-spazio evita duplicati; aggiunta e rimozione aggiornano immediatamente catalogo, dettaglio e pagina Preferiti e persistono dopo reload.

9.7. Segnalazioni

Dal dettaglio di uno spazio l'utente seleziona categoria, descrive il problema e allega facoltativamente una foto. Il server assegna priorit  alta a problemi tecnici e di accessibilit , media alla pulizia e bassa ad altro. L'utente consulta ma non modifica le proprie segnalazioni. L'admin aggiorna open -> in_progress -> resolved e l'autore riceve una notifica persistente.

9.8. Notifiche e avvisi

Le notifiche sono personali, persistenti e non eliminabili manualmente. Possono essere lette singolarmente o tutte insieme. Gli avvisi globali generano, nella stessa transazione, una notifica per ogni utente normale esistente in quel momento; admin e iscritti successivi sono esclusi.

9.9. Gestione amministrativa

9.9.1. Edifici e spazi

L'admin crea, modifica ed elimina edifici e spazi. Per uno spazio gestisce edificio, piano, tipologia, capienza, accessibilit , servizi e stato. Una riduzione sotto i posti impegnati viene bloccata. Manutenzione, disattivazione o eliminazione coordinano cancellazioni e notifiche.

9.9.2. Disponibilit  e indisponibilit 

Le fasce sono ricorrenti per giorno della settimana e periodo. Le sovrapposizioni sono vietate, mentre fasce consecutive sono ammesse. La modifica ritira la configurazione precedente e crea una nuova versione; le fasce ritirate possono essere mostrate e riutilizzate. Le indisponibilit  eccezionali coprono un intervallo di una data precisa e cancellano le prenotazioni future coinvolte.

9.9.3. Prenotazioni, utenti e segnalazioni

L'admin consulta organizzatore, partecipanti, check-in e storico, ma non crea prenotazioni per gli studenti. Puo cercare gli utenti ed eliminare soltanto account normali. Le segnalazioni mostrano stato attuale, priorit , autore e foto protetta.

9.9.4. Statistiche

Il periodo Dal-Al alimenta dati calcolati dal database. Prenotazioni indica i gruppi completati. Il tasso di utilizzo e presenze diviso posti offerti; il completamento check-in e presenze diviso partecipanti prenotati. Si usano soltanto fasce terminate e offerte, includendo quelle senza prenotazioni ed escludendo future, in corso e indisponibili. I denominatori zero producono N/D.

Indicatore	Formula
Prenotazioni	Numero di gruppi nelle fasce terminate del periodo.
Tasso di utilizzo	Somma presenze / somma posti offerti x 100.
Check-in completati	Somma presenze / somma partecipanti prenotati x 100.
Assenze	Partecipanti prenotati - presenze registrate.

10. Sicurezza, configurazione e deployment

10.1. Password e sessioni

Le password devono avere 8-64 caratteri, maiuscola, minuscola, cifra, speciale e nessuno spazio. Sono hashate con script (N=16384, r=8, p=1), sale casuale di 16 byte e chiave di 64 byte. Gli hash malformati sono rifiutati prima del calcolo. Il confronto usa timingSafeEqual.

Il JWT HS256 contiene soltanto sub, role, iat, exp e jti. Ogni richiesta protetta verifica anche account, ruolo e sessione corrente nel database. Il refresh è casuale, nel database ne resta solo SHA-256 e ogni rotazione invalida immediatamente i token precedenti.

10.2. Variabili di ambiente

Variabile	Uso
NODE_ENV	development, test o production.
HOST / PORT	Interfaccia e porta Express; valori locali predefiniti 127.0.0.1:3000.
SLOTLAB_JWT_SECRET	32 byte casuali rappresentati da 64 caratteri esadecimali.
SLOTLAB_DB_PATH	Percorso assoluto del database scelto.
SLOTLAB_UPLOAD_DIR	Cartella assoluta e persistente delle foto delle segnalazioni.
SLOTLAB_TRUST_PROXY	none oppure loopback quando Nginx è sullo stesso host.
SLOTLAB_SMTP_*	Host, porta, TLS, mittente, credenziali e CA del relay email.
SLOTLAB_ADMIN_*	Dati temporanei usati dal seed per creare l'amministratore.

10.3. Sviluppo, test e produzione

- **Sviluppo:** Angular/Ionic serve il frontend e il proxy inoltra /api al backend locale.
- **Test:** i test backend creano database temporanei; Angular usa TestBed, HttpTestingController e jsdom.
- **Produzione:** la build genera **www**; Nginx serve i file, applica il fallback alla SPA e inoltra /api a Express sul loopback.
- **HTTPS:** è necessario per il cookie Secure e per un accesso mobile affidabile. Il certificato deve essere riconosciuto dal dispositivo.
- **Persistenza:** database e upload devono risiedere fuori dalla directory pubblica e sopravvivere ai riavvii.

10.4. Comandi principali

```
# Backend
cd backend
npm run db:migrate
npm run db:seed
npm start

# Frontend
cd ..
npm run start:https

# Verifica automatica
npm test -- --prefix backend
npm test -- --watch=false
npm run lint
npm run build -- --configuration production
```

11. Verifica e qualita

11.1. Strategia di test

Il backend usa test unitari e HTTP con server e database temporanei. Verifica migrazioni, integrita, autenticazione, concorrenza, prenotazioni, partecipanti, capienza, tempo, account, foto, statistiche e risposte di errore. Il frontend usa test di componenti e servizi con chiamate HTTP simulate per controllare rendering, routing, aggiornamento degli elenchi, sessione e casi vuoti.

11.2. Collaudo manuale

La checklist di collaudo accompagna questa relazione e copre tutti i flussi utente e admin, reload e persistenza, mobile e desktop, tema chiaro/scuro, tastiera, errori di rete e configurazione finale. Le prove distruttive vengono eseguite per ultime e sempre su una copia separata del database.

11.3. Responsive design

Le schermate sono progettate per desktop, tablet e smartphone. A 900 px l'amministrazione passa dalla sidebar alla navigazione mobile. Tabelle, elenco disponibilita e grafici hanno scorrimento interno; card e form riducono le colonne. Il collaudo previsto usa larghezze da 320 a 1440 px e almeno un telefono fisico.

11.4. Limiti operativi dichiarati

- Il QR identifica lo spazio ma non dimostra da solo la presenza fisica: l'identita e la finestra temporale sono comunque verificate dal server.
- Il recupero invia una nuova password via email come requisito didattico; non e un flusso a link monouso.
- Le statistiche storiche iniziano quando vengono consolidati gli snapshot: non ricostruiscono automaticamente periodi precedenti privi di dati.
- L'eliminazione definitiva di spazi o edifici rimuove anche gli snapshot collegati, secondo le regole del progetto.
- OpenStreetMap e SMTP richiedono connettivita e configurazione esterna per le relative funzioni.

12. Conclusioni

SlotLab trasforma un prototipo grafico in un sistema completo per la gestione di posti universitari. La separazione dei ruoli, le transazioni sulle prenotazioni, la sessione revocabile, i controlli temporali e gli snapshot statistici garantiscono coerenza tra esperienza utente e amministrazione.

Gli obiettivi funzionali risultano coperti dall'architettura implementata: catalogo del campus, disponibilit  reale, prenotazioni nominative, partecipanti, check-in, preferiti, segnalazioni, notifiche, amministrazione e statistiche. La dichiarazione finale di prontezza richiede il superamento della checklist manuale sul database di collaudo e la verifica HTTPS/SMTP nell'ambiente di destinazione.

12.1. Riferimenti interni

- docs/api-contract.md - contratto delle API e regole funzionali.
- docs/relational-schema.md - modello logico e decisioni sul database.
- backend/db/integrity-rules.md - separazione tra vincoli SQLite e controlli Express.
- backend/README.md e backend/db/README.md - avvio, migrazioni, seed e configurazione.
- docs/deployment.md - configurazione di pubblicazione sullo stesso dominio.
- docs/checklist-collaudo-manuale.md - matrice completa di verifica manuale.